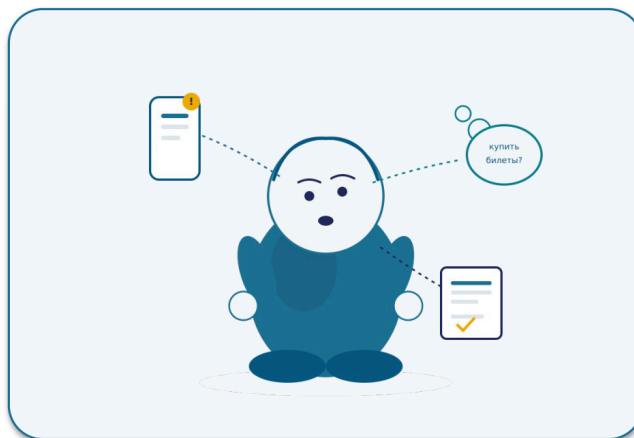


Пока вы читаете это предложение — на что вы НЕ обращаете внимания?

Ваш мозг каждую секунду выбирает, что важно, а что — фон.



Механизм, который сегодня разберём внутри модели ИИ, называется точно так же — «внимание». Он работает похоже — но не так, как ваш.

1 / 35

Начнём не с AI, а с вас. Пока вы читаете это предложение, ваш мозг непрерывно решает маленькую, но важную задачу: что сейчас важно держать в фокусе, а что можно оставить фоном. Где-то на периферии — уведомление на телефоне, недодуманная мысль про вечерние планы, ощущение неудобного стула. Всё это конкурирует за один и тот же ограниченный ресурс — ваше внимание. Вы не осознаёте выбор в моменте, но он происходит постоянно. На слайде — персонаж, которого одновременно тянут в три стороны: уведомление, посторонняя мысль, незакрытая задача. Это не про рассеянность как недостаток — это нормальная работа механизма внимания. У человека внимание ограничено: мы физически не можем одинаково полно обрабатывать всё одновременно, поэтому мозг постоянно расставляет приоритеты — что в фокус, что в фон. Сегодняшняя лекция — про то, что происходит внутри AI-модели между вашим запросом и её ответом. И один из четырёх внутренних этапов, которые мы разберём, называется буквально так же, как то, что вы только что почувствовали — ****механизм внимания****. Когда модель обрабатывает ваш текст, она тоже решает, на какие части входа «смотреть» внимательнее, а какие оставить фоном. Важная оговорка, к которой мы вернёмся в Разделе 3: сходство здесь — по названию и по общей идее «выбора приоритета», а не по устройству. Внутри модели это не психологический процесс, а конкретная вычисляемая операция — матрица весов, которую можно посчитать и посмотреть. Разберём это подробно, когда дойдём до сути. Прежде чем туда дойти — пройдем весь путь запроса через модель: токенизацию, эмбединги, внимание, сэмплинг. У каждого этапа — своё конкретное инженерное следствие для того, как вы работаете с AI на практике.

ЛЕКЦИЯ 2

Как работают современные большие модели

4 этапа inference: токенизация · эмбеддинг · внимание · сэмплинг



2 / 35

Сегодняшняя лекция — вторая в курсе. Лекция 1 заложила слоистую картину «модель → чат → агент → приложение» и описала самый нижний слой, модель, как stateless-инференс: на вход данные, на выход предсказание, без памяти между вызовами. Этого описания было достаточно, чтобы говорить, где именно используется модель в большей системе, но недостаточно, чтобы говорить, что внутри этой модели происходит. Сегодня мы аккуратно разбираем чёрный ящик на четыре этапа. Дорожная карта внизу слайда соответствует этим четырём этапам: токенизация, эмбеддинги, механизм внимания, сэмплинг. Каждому посвящён отдельный раздел. После четырёх разделов мы возвращаемся к трём практическим вопросам, которые Лекция 1 оставила без ответа: почему промпт с ролью работает лучше пустого, почему AI плохо считает буквы, почему один и тот же запрос даёт разные ответы. На все три ответим через конкретный внутренний механизм, не через интуицию. Тон лекции — объяснительно-инженерный. Мы не описываем устройство трансформера во всей полноте: формулы внимания, матрицы Q/K/V, многоголовое внимание, позиционное кодирование — останется за рамками лекции. Студенту, которому нужна полная техническая картина, дальше дорога в курсы по deep learning и в каноническую статью Vaswani et al. (2017). Здесь мы целенаправленно держим уровень детализации такой, чтобы каждый внутренний механизм связывался с практикой — с тем, как вы будете ставить задачи модели в своей инженерной работе.

Карта лекции — 6 разделов



3 / 35

Прежде чем погружаться в механику больших моделей — короткая карта маршрута. Лекция состоит из шести разделов, и каждый отвечает на свой вопрос. Раздел 0, в котором мы сейчас, — введение: что такое токен, краткий ресар Лекции 1 и центральный вопрос, ради которого мы здесь. Раздел 1 — токенизация. Как модель «видит» ваш текст. Что такое токен, как работает BPE, почему strawberry дробится на три части, и почему русский текст стоит модели в два раза дороже английского. Раздел 2 — эмбединги. Пространство смыслов: как слова превращаются в векторы, что такое семантическое сходство, и как из этой математики получается семантический поиск и мост к RAG. Раздел 3 — внимание. Как модель решает, на какие токены смотреть сейчас. Разберём механизм внимания на конкретном примере и поймём, почему промпт с ролью «ты эксперт» действительно меняет ответ. Раздел 4 — сэмплинг. От распределения вероятностей к одному токену. Температура, top-p, top-k, max_tokens — четыре ручки API, которые вы будете крутить каждый раз, когда вызываете LLM. Раздел 5 — финал. Закрытие трёх «почему», поставленных в центральном вопросе. Сравнение ML и LLM как двух парадигм, отличия Human vs AI, домашнее задание к семинару и анонс Лекции 3. Держите эту карту в голове — она поможет не потеряться в деталях. После каждого раздела мы будем останавливаться и фиксировать главное.

Сегодня углубляем слой «модель» из четырёх слоёв Лекции 1



4 / 35

Чтобы не терять контекст, на одну минуту вернёмся к Лекции 1. На той лекции мы построили слоистую картину работы AI в продукте: внизу модель, над ней чат-цикл, над чатом — агентный цикл, и сверху — приложение, в которое всё это упаковывается. Лекция 1 описала самый нижний слой, модель, как stateless-функцию: на вход данные, на выход предсказание, и никакой памяти между двумя вызовами. С этим описанием мы умеем говорить о том, **где** модель находится в большей системе, но мы оставили в стороне вопрос — **что** именно происходит внутри этой stateless-функции, когда вы пишете в чат «Сегодня я съел яблоко» и получаете ответ. Сегодня мы аккуратно открываем этот чёрный ящик. Главное содержание лекции — четыре внутренних этапа, через которые проходит ваш запрос между моментом отправки и моментом получения ответа. Эти четыре этапа: токенизация (как модель разрезает текст на единицы, с которыми умеет работать), эмбединг (как каждой единице сопоставляется численное представление смысла), механизм внимания (как модель решает, на что опираться в текущем контексте), сэмплинг (как из распределения вероятностей выбирается один конкретный токен). На Лекции 1 модель называлась «чёрным ящиком stateless inference». Сегодня мы перестаем держать его чёрным. После этой лекции в этом ящике останутся уже не четыре механизма, а четыре отдельных этапа с понятной ролью каждого — и с конкретными инженерными следствиями для того, как вы будете ставить задачи модели.

Главный вопрос лекции

«Что происходит внутри LLM между моим запросом и ответом — и какие из этих механизмов меняют, как я её использую?»

3 ответа — финал лекции

1

Почему промпт с ролью работает лучше пустого?

→ Раздел 3 — внимание

2

Почему AI плохо считает буквы?

→ Раздел 1 — токенизация

3

Почему один запрос даёт разные ответы?

→ Раздел 4 — сэмплинг

5 / 35

У сегодняшней лекции один центральный вопрос, и он сформулирован коротко: что происходит внутри LLM между моим запросом и ответом, и какие из этих внутренних механизмов меняют, как я её использую. Заметьте, что вопрос двусоставный. Первая часть — описательная: понять, что технически устроено внутри. Вторая часть — прагматическая: из всего внутреннего устройства выделить то, что меняет инженерную практику. Если внутренний механизм не влияет на то, как вы ставите задачу модели или интерпретируете её ответ, мы его сегодня не разбираем. В ответе на этот вопрос участвуют три обещания, которые Лекция 1 оставила без ответа. Первое — почему промпт с ролью работает лучше пустого. Тот, кто пробовал, знает, что «Ты эксперт по Python, объясни асинхронность джуниору» даёт качественно другой ответ, чем просто «Объясни асинхронность». Сегодня мы поймём механизм этого эффекта на уровне внимания. Второе — почему AI плохо считает буквы. Классический пример со словом `strawberry` и подсчётом буквы `r` — не баг и не пробел в обучении, а структурное следствие того, как устроена токенизация. Сегодня разберём это в Разделе 1. Третье — почему один и тот же запрос может давать разные ответы. Источник — стохастический сэмплинг при температуре больше нуля, и в Разделе 4 мы увидим механизм. Эти три обещания работают как якоря лекции: к ним мы будем возвращаться в каждом из четырёх разделов, и в самом конце они получат явный ответ — три «почему» через конкретные внутренние механизмы.

Раздел 1

Токенизация

Как модель видит ваш текст

0 Введение

1 Токены

2 Эмбединги

3 Внимание

4 Сэмплинг

5 Финал

6 / 35

Введение закончено: мы зафиксировали центральный вопрос — что внутри LLM меняет то, как вы её используете, — и три промиса-якоря, на которые получим ответы к финалу. Дальше — первый из четырёх внутренних этапов inference: токенизация. Это самый ближний к пользователю слой; именно с ним сталкивается ваш текст, как только вы нажимаете «отправить». Текст не попадает в модель буквами и не попадает словами. Он сначала разрезается на токены — короткие подпоследовательности, которые модель «знает» из своего обучающего корпуса. В этом разделе четыре слайда. Сначала мы зафиксируем базовое определение токена через три коротких примера на английском и русском. Потом — почему модель использует BPE как компромисс между алфавитом и словарём, и одна инженерная деталь, важная для понимания цены: словарь строится один раз перед обучением, в inference происходит только lookup. Дальше — самый показательный случай слепоты к буквам: «сколько r в strawberry», и через него — три класса задач, в которых LLM ошибается систематически. И завершим раздел сравнением стоимости одного и того же текста на разных языках — почему русский дороже английского примерно в два раза, и что с этим делать в production. Темп в разделе держим практический: на каждом слайде есть проверяемая инженерная импликация.

Поток данных в LLM — туда и обратно

Каждый из этапов разберём отдельно. Сейчас — карта целиком.



7 / 35

Прежде чем углубляться в детали каждого этапа, посмотрим на полную картину — что происходит между вашим запросом и ответом модели. Это карта, к которой мы будем возвращаться: с правой стороны каждого следующего раздела вы будете видеть, какая именно стадия этого pipeline сейчас в фокусе. Слева — ваш входной текст: например, `«Привет, мир»`. Первое преобразование — **токенизация**: текст разрезается на единицы из словаря модели; `«Привет»` превращается в, условно, `[Прив][ет]`. Эти токены — целые числа, id из словаря. Это первый из четырёх этапов, и именно ему посвящён ближайший Раздел 1. Следующий шаг — **embedding lookup**: каждому id токена сопоставлен выученный вектор в памяти модели. Слово исчезает; остаётся последовательность векторов. Это входит в Раздел 2. Дальше — **LLM-инференс**: серия слоёв с механизмом внимания, который мы разберём в Разделе 3. Здесь модель «думает» — но думает она над векторами, не над словами. Из этой обработки на выходе появляется ещё один вектор — кандидат на следующий токен. Этот вектор-кандидат превращается в **распределение вероятностей** по всему словарю модели: «насколько правдоподобно, что следующий токен — этот? а этот? а вот этот?». Из распределения выбирается **один токен** через сэмплинг — это Раздел 4. И последний шаг — **обратное преобразование**: id токена через де-токенизацию превращается обратно в текст. Этот шаг почти симметричен токенизации в самом начале, и обычно его относят к тому же первому этапу. Главное наблюдение для практики: «слово» появляется только на границах — на входе и на выходе. Всё, что происходит внутри модели — это операции над векторами. Сегодняшняя лекция — про эти внутренние операции, без формул, но с инженерным пониманием, что и где происходит.

Токен — id из словаря модели. Не буква и не слово.

Статистически частая подпоследовательность

Пример 1.	cat	→	[cat]	1 токен / 1 id
Пример 2.	tokenization	→	[token] [ization]	2 токена
Пример 3.	клубника	→	[к] [луб] [ника]	3 токена · o200k_base

В среднем: 1 токен ≈ 4 символа в EN ≈ 2 символа в RU

Для русского inference обходится примерно вдвое дороже — мы вернёмся к этому через 2 слайда.

8 / 35

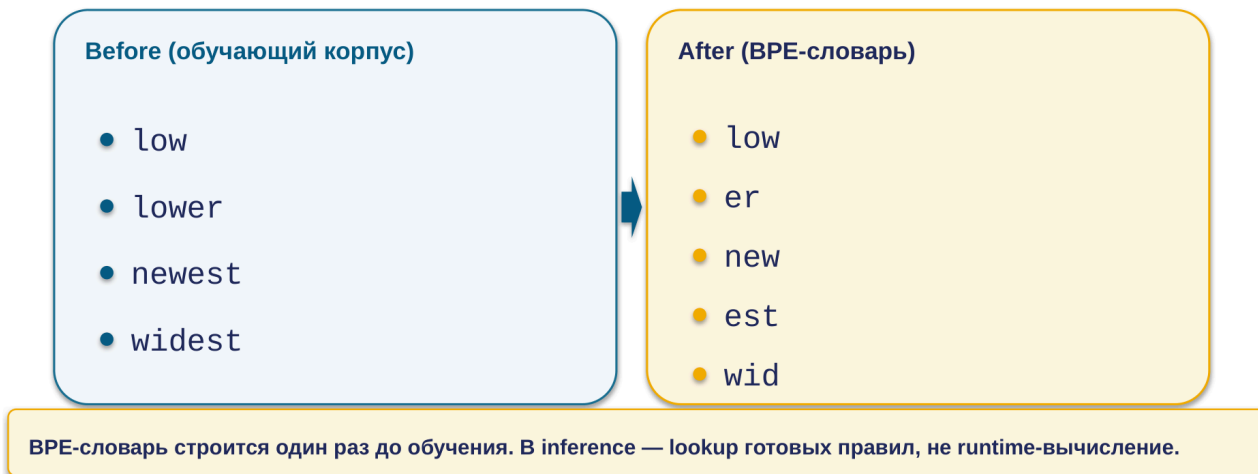
Главный факт первого раздела: LLM видит ваш запрос не буквами и не словами, а токенами. Токен — это идентификатор, целое число из словаря модели; этот словарь зафиксирован при обучении и не меняется в момент использования. Когда вы пишете в чат «Сегодня я съел яблоко», ваш текст сначала разрезается на токены, каждый токен превращается в число — id из словаря — и дальше модель работает только с этой последовательностью чисел. Для удобства мы записываем токены в квадратных скобках, но в памяти модели это всегда числа. На слайде — три примера, иллюстрирующих основное наблюдение. Короткое распространённое английское слово `cat` попадает в словарь как единый токен. Слово `tokenization` распадается на два токена: общая основа `[token]` и часто встречающийся суффикс `[ization]`. Слово `клубника`, протестированное на токенизаторе GPT-4o `o200k\_base`, разрезается на три токена. Закономерность простая: чем чаще подпоследовательность встречалась в обучающем корпусе, тем выше шанс, что она попадёт в словарь целиком; редкие или специфические для других языков последовательности дробятся мельче. Из этого следует практический ориентир, которым пользуются инженеры на этапе оценки стоимости API: в среднем один токен соответствует примерно четырём символам английского текста или примерно двум символам русского. Это не точная формула, а порядок, но его достаточно для большинства бюджетных оценок. Полезное упражнение: попробуйте угадать, на сколько токенов разрежется слово `сильнее` — на один, два или три? Ваша интуиция может ошибиться: для русского даже короткое слово часто получается из 2-3 токенов. Точный ответ можно проверить на сервисе Tiktokenizer. Маленькое следствие, важное для практики: на числах токенизация ведёт себя непредсказуемо. Строка `1234567` может быть разрезана как `[123][4567]`, `[12][345][67]` или ещё как-то — в зависимости от того, какие подпоследовательности цифр часто встречались в

корпусе. Для арифметики это означает: чистый LLM-инференс — плохой калькулятор; правильный паттерн — внешний инструмент, Code Interpreter или Python sandbox.

BPE — компромисс между алфавитом и словарём

Словарь не из всех слов (как лемматизация) и не из всех букв (как character-level) — а из частых подпоследовательностей.

Словарь строится один раз перед обучением; в inference — lookup готовых правил.



[1] Sennrich et al. (2016) — NMT of Rare Words / BPE

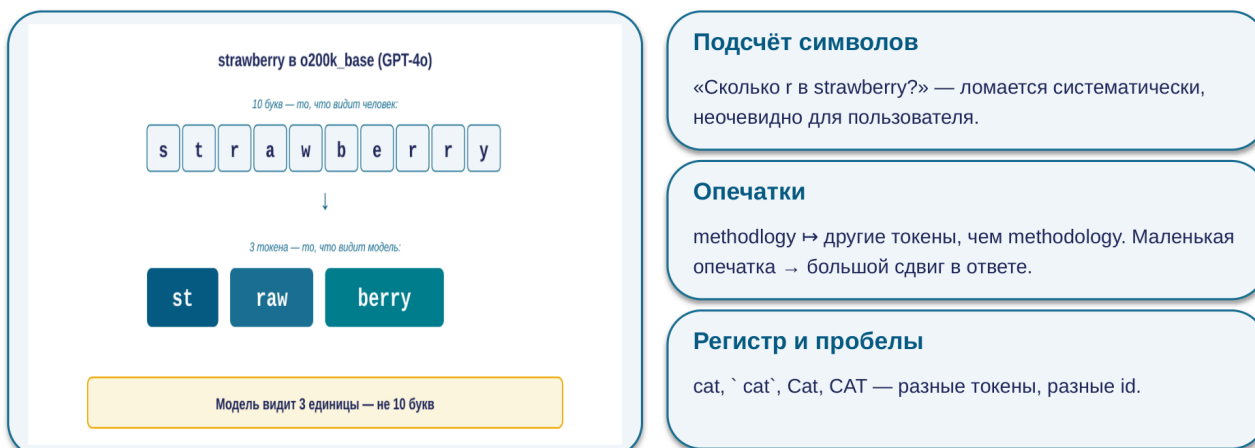
9 / 35

Алгоритм, которым строятся словари большинства современных LLM, называется BPE — Byte-Pair Encoding. По сути, это компромисс между двумя крайностями. Можно было бы взять словарь отдельные символы — тогда любой текст представим, но последовательности получаются очень длинные, и модели тяжело учить долгие зависимости. Можно было бы, наоборот, взять словарь целые слова — тогда последовательности короткие, но любое незнакомое слово, опечатка или имя собственное становится ``, и модель не способна с ним работать. BPE сидит ровно посередине: словарь состоит из подпоследовательностей разной длины, которые алгоритм нашёл как самые частые в обучающем корпусе. Идея алгоритма проста: начинаем с алфавита из отдельных символов и итеративно объединяем пары, которые встречаются вместе чаще всего, добавляя пару в словарь как новый «составной» символ. На небольшом учебном корпусе из слов `low / lower / newest / widest` BPE может выучить словарь вроде `low / er / new / est / wid` — отдельно частые корни и отдельно частые суффиксы. После многих итераций на большом корпусе получаются словари в десятки и сотни тысяч записей. Важная инженерная деталь, которая часто упускается, — это **момент построения словаря**. BPE-словарь строится один раз, до обучения модели. Это шаг подготовки данных: на корпусе обучения запускают BPE-алгоритм, фиксируют итоговый словарь и набор merge-правил, и после этого ничего не меняется. В момент работы модели токенизация — это lookup готовых правил, а не runtime-вычисление. Когда вы отправляете запрос в API, текст разрезается за миллисекунды по предзаписанной таблице. Современных алгоритмов для подсловарных токенизаторов на практике три: BPE используется в GPT-семействе и в Llama 3+; WordPiece — в BERT и потомках; SentencePiece — в Llama 2, Mistral, T5 и многих open-weight моделях. Технически они отличаются способом выбора пар и обработкой пробелов, но для

пользователя LLM эти различия не критичны — нужно лишь помнить, что токенизатор поставляется вместе с моделью и его нельзя «поменять на ходу».

Источники: [1] Sennrich et al. (2016) — NMT of Rare Words / BPE — BPE — компромисс: словарь из частых подпоследовательностей, не букв и не слов.
<https://arxiv.org/abs/1508.07909>

AI ошибается в «сколько r в strawberry» — слова не из букв, а из 3 токенов



Буквы: внешний инструмент (Python, regex) или посимвольный запрос. Топ-модели 2026 сами вызывают код вместо одного прохода.
Числа: тоже режутся непредсказуемо (1234 → 12+34, но 55688 → 556+88) → GPT-4: 59% точности на 3-знач. умножении, 4% на 4-знач., 0% на 5-знач. без калькулятора.

[1] Counting Ability of LLMs & Impact of Tokenization (2024)

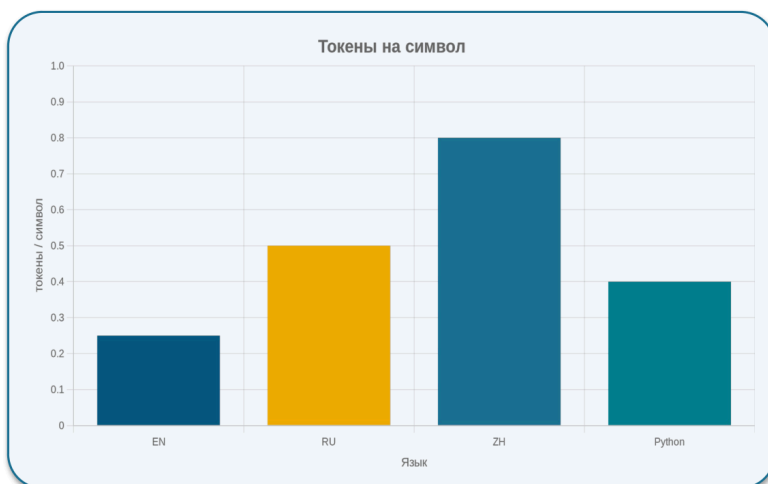
10 / 35

Классическая ошибка LLM, которую можно воспроизвести почти любой моделью: на вопрос «сколько букв `r` в слове `strawberry`» модель часто отвечает «два». Правильный ответ — три. Почему так получается? Слово `strawberry` в современных tiktoken-токенизаторах разрезается на три токена: `[st]`, `[raw]`, `[berry]`. На входе в модель — три числовые единицы, не десять букв. Когда модель «отвечает» на вопрос о количестве буквы `r`, она не имеет прямого доступа к посимвольному представлению; она работает с тремя id, для каждого из которых в её памяти лежит выученный вектор. Внутри этих векторов нет явного списка «вот тут стоит буква `r` на позициях 3 и 8» — внутри векторов лежит статистическая информация о том, в каких контекстах токен `[raw]` встречается. ****Модель видит токены, не буквы.**** Это явление мы будем называть ****слепотой к буквам****. Это не баг и не плохое обучение — это прямое структурное следствие того, как устроена токенизация. Из той же природы вытекают три практических следствия. Первое — подсчёт символов: задачи такого вида ломают LLM не всегда, но систематически и неочевидно для пользователя. Второе — точная обработка опечаток: когда модель видит слово с опечаткой, оно может разрезаться на совершенно другие токены, чем правильное слово, и модель «видит» иной набор id; это объясняет, почему мелкие опечатки иногда вызывают неожиданно большие изменения в ответе. Третье — чувствительность к регистру и пробелам: `cat`, `cat` с ведущим пробелом, `Cat`, `CAT` могут оказаться разными токенами с разными выученными векторами. Инженерный вывод: если ваша задача требует точной посимвольной операции — подсчёта символов, поиска подстроки по точному совпадению, преобразования регистра, проверки соответствия регулярному выражению — не делайте её чистым LLM-инференсом. Используйте внешний инструмент: код в Python sandbox, регулярное выражение, специализированный микросервис. Современные

топ-модели (o1-класс, Claude 4.7, GPT-5) часто отвечают на `strawberry` правильно — но не потому, что один forward-pass через нейросеть умеет посимвольно работать со строкой, а потому, что модель внутри вызывает Python-инструмент или генерирует явный пошаговый подсчёт. Структурный факт «AI не видит букв, видит токены» при этом не меняется. Та же природа объясняет вторую, менее очевидную проблему — арифметику. Числа тоже проходят через токенизатор, и разбиение на токены непредсказуемо совпадает с разрядами: `1234` может разрезаться на `12` + `34` (совпадает с сотнями), а `55688` — на `556` + `88` (уже не совпадает). Модель не видит цифры по отдельности — она видит эти нерегулярные группы. Следствие измеримо: на 3-значном умножении GPT-4 без внешних инструментов даёт около 59% точности, на 4-значном — уже 4%, а на 5-значном — 0% (arXiv 2410.19730). Решение то же самое, что и для букв: внешний калькулятор или код, либо явный пошаговый вывод, а не один прямой проход модели. Источники: [1] Counting Ability of LLMs & Impact of Tokenization (2024) — GPT-4 без калькулятора: 59%/4%/0% на 3-/4-/5-значном умножении — тот же tokenizer-cut механизм. <https://arxiv.org/abs/2410.19730> [VFY-day-of]

Один и тот же текст по-русски стоит в 2× дороже, чем по-английски

Токены на символ — следствие распределения языков в обучающем корпусе



Ориентир токены/символ

Английский	0.25
Русский (gold)	0.50
Китайский	0.80
Python-код	0.40

API-стоимость RU $\approx$ 2× от EN. Для batch — переводить в EN, если допустимо.

11 / 35

Одно и то же текстовое сообщение в API стоит разное число токенов в зависимости от языка. Это прямое следствие того, что BPE-словарь учится на корпусе, в котором английский язык представлен заметно шире других. Для английского много частых подпоследовательностей укладывается в одиночные токены; для русского, китайского, японского, кода — токенов в среднем больше на тот же по смыслу текст. Ориентировочные цифры в токенах на символ для современных токенизаторов GPT-семейства. Для естественного английского — около 0.25 токена на символ: текст из 100 символов занимает примерно 25 токенов. Для русского — около 0.5 токена на символ: тот же по смыслу текст из 100 символов занимает примерно 50 токенов. Для китайского — около 0.8 токена на символ, потому что один иероглиф часто соответствует одному или нескольким токенам. Для Python-кода — около 0.4 токена на символ. Эти числа — порядок, не точная формула: разные источники дают разброс от 1.5× до 2.5× в зависимости от характера текста и версии токенизатора. Инженерное следствие важно: обращение к API на русском обходится примерно в два раза дороже, чем на английском, за один и тот же по смыслу запрос. Если у вас batch-задача — массовая обработка тысяч документов — и предметная область допускает работу на английском, имеет смысл перевести входы и выходы на английский для экономии. Это не означает «всегда работать на английском»: для большинства интерактивных задач удобство и качество ответа важнее разницы в стоимости. Но при больших объёмах это значимый фактор бюджета. Дополнительное следствие — контекстное окно тоже расходуется неравномерно. Контекстное окно у современных моделей измеряется в токенах, не в словах и не в символах. Документ объёмом 80 тысяч символов на английском уместится в 20 тысяч токенов; тот же документ на русском займёт примерно 40 тысяч. Для модели с окном 32 тысячи второй вариант не поместится. Цифра «в 2 раза дороже»

относится к актуальным токенизаторам GPT-семейства; для специализированных моделей с увеличенной долей русского в обучающих данных (YandexGPT, GigaChat) разрыв может быть меньше.

Раздел 2

Эмбединги

Пространство смыслов

0 Введение

1 Токены

2 Эмбединги

3 Внимание

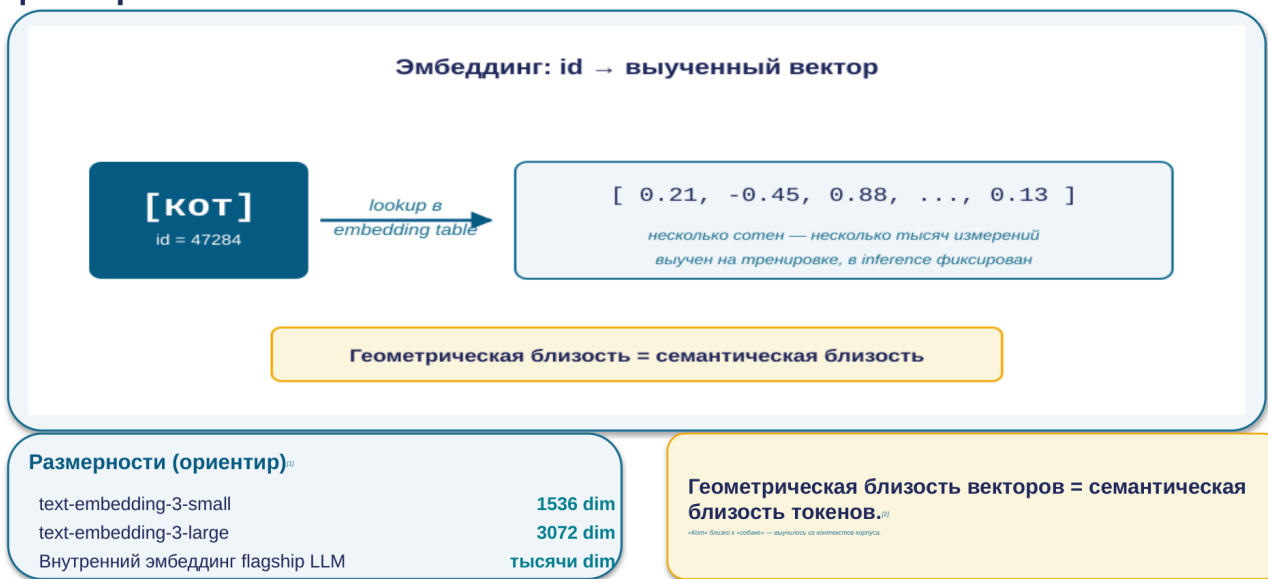
4 Сэмплинг

5 Финал

12 / 35

Мы прошли первый этап — токенизация. Текст превращается в последовательность id из словаря модели. Дальше — второй этап, эмбединги. Это шаг, на котором каждый id из словаря заменяется на вектор в многомерном пространстве: для маленьких embedding-моделей это 384 или 1536 измерений, для внутренних слоёв больших LLM — тысячи. И ключевое свойство этого пространства, которое было выучено на тренировке: геометрическая близость векторов отражает семантическую близость токенов и фрагментов. Эмбединги — это не интересный сам по себе слой; модель внутри использует их как промежуточный шаг к вниманию и сэмплингу. Зачем нам про них говорить отдельно? Потому что на них стоит целое прикладное направление: similarity, кластеризация и семантический поиск. И семантический поиск — это базовый слой RAG, Retrieval-Augmented Generation, к которому мы вернёмся в Лекции 3. В этом разделе четыре слайда плюс этот разделитель. Сначала зафиксируем, что такое эмбединг через схему «токен → вектор». Потом разберём sentence similarity — близость на уровне предложений на современных embeddings 2026 года, не только на уровне слов. Затем три практических применения: similarity, clustering, search, — карта для последующих лекций. И завершим разделом сравнением семантического поиска с full-text поиском на простом запросе «клубника» — что находит один, а что другой.

Каждому токenu сопоставлен вектор — выучен на тренировке, фиксирован



[1] OpenAI — Embeddings API · [2] Mikolov et al. (2013) — word2vec

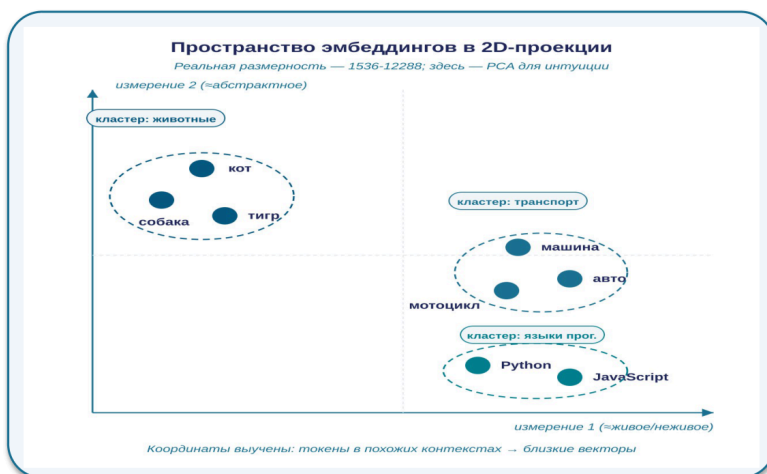
13 / 35

Когда мы сказали «токен — это id из словаря», осталась открытая часть: как модель работает с этим id. Прямо с числом она не работает — между токенами нет осмысленной арифметики, и модели нужен способ представить «значение» каждого токена в форме, пригодной для нейросети. Этот способ — **эмбединг**, или **векторное представление**. Каждому токenu словаря модели сопоставлен вектор фиксированной длины — список чисел с плавающей точкой. Для токена `[кот]` в гипотетической модели вектор может выглядеть как `[0.21, -0.45, 0.88, ..., 0.13]` — несколько сотен или несколько тысяч чисел. Вектор не назначается вручную; он выучивается на этапе обучения модели вместе со всеми остальными весами нейросети. Когда обучение заканчивается, таблица «токен → вектор» — embedding table — фиксируется и в дальнейшем не меняется. На inference модель просто делает lookup: получает id токена, забирает соответствующий вектор из таблицы, передаёт в следующий слой. Размер вектора — гиперпараметр конкретной модели. Несколько публичных ориентиров для специализированных embedding-моделей OpenAI: `text-embedding-3-small` — 1536 измерений; `text-embedding-3-large` — 3072 измерения. Размерности внутренних embedding-таблиц у flagship-LLM (GPT-4, Claude и т.п.) официально не публикуются; по неофициальным оценкам их порядок — несколько тысяч до десяти с лишним тысяч измерений. Для целей этой лекции важен не точный размер, а порядок: тысячи измерений. Главное свойство эмбедингов: геометрическая близость в этом пространстве соответствует смысловой близости. «Кот» близко к «собаке»; «врач» близко к «терапевту»; «SSL» близко к «HTTPS». При этом модель никем не была явно обучена «кот и собака — одна категория» — это свойство возникло само из того, что в обучающем корпусе оба слова часто появлялись в похожих контекстах. Близость в эмбединг-пространстве — это статистическое отражение того, как слова

используются, не семантический справочник, написанный человеком. Источники:
[1] OpenAI — Embeddings API — публичные эмбединги: text-embedding-3-small 1536, large 3072 измерения. <https://platform.openai.com/docs/guides/embeddings>
[VFY-day-of] [2] Mikolov et al. (2013) — word2vec — исторический контекст: геометрическая близость = смысловая близость. <https://arxiv.org/abs/1301.3781>

Пространство эмбедингов — где живут векторы

Перед тем как сравнивать векторы — посмотрим, в каком пространстве они находятся.



1 Размерность

OpenAI text-embedding-3-small: 1536 dim.
 text-embedding-3-large: 3072 dim.
 Flagship LLM: тысячи.

2 Обучение

Координаты не задаются вручную. Модель учится:
 токены в похожих контекстах → близкие векторы.

3 Проекция

Чтобы увидеть пространство — PCA или t-SNE
 снижают размерность до 2D/3D.
 Часть структуры теряется.

Семантическая близость = геометрическая близость векторов. На следующем слайде — мера.

14 / 35

Прежде чем перейти к sentence similarity, важно понять, в каком именно пространстве живут эмбединги. Это первый момент в лекции, когда мы говорим про «многомерные пространства», и интуиция здесь обычно сбивает. Когда мы говорим «пространство эмбедингов», мы имеем в виду математическое пространство — векторное пространство, в котором каждой точкой является один вектор. Этот вектор состоит из чисел: 1536 чисел для OpenAI `text-embedding-3-small`, 3072 для `text-embedding-3-large`, и тысячи для внутренних эмбедингов flagship-моделей. Каждое число — это координата по своему «направлению» или «измерению». В отличие от трёхмерного пространства, к которому мы привыкли по физическому миру, у этих измерений нет очевидного смысла «вверх-вниз» или «вправо-влево». Это абстрактные направления, которые модель выучила в процессе обучения. Что значит «выучила»? Цель обучения — устроить пространство так, чтобы токены, которые в обучающем корпусе появлялись в похожих контекстах, лежали близко друг к другу. Слова `кот` и `собака` встречаются рядом со словами типа `кормить`, `гладить`, `питомец` — и в результате их эмбединги получаются близкими. То же самое со словами `Python` и `JavaScript` — они появляются рядом со словами `код`, `функция`, `переменная`. Эта структура возникает не из правил, а из статистики: модель видела миллионы примеров и подстроила числа так, чтобы геометрия пространства отражала семантику языка. Чтобы такое пространство хоть как-то увидеть — нужна **проекция**. На слайде слева — упрощённая 2D-проекция: 8 точек, разделённых на три смысловых кластера. Реальная проекция получается из 1536-мерного пространства через алгоритмы PCA (главных компонент) или t-SNE; они выбирают такие две оси, на которых видно максимум различий между точками. Но любая 2D-проекция теряет часть информации — реальная структура многомерна, и красиво нарисовать её на

плоскости невозможно. Главное практическое следствие: семантическая близость = геометрическая близость векторов. Далее, на следующем слайде, мы измерим эту близость числом — через cosine similarity — и увидим, что это работает не только на словах, но и на целых предложениях.

Близость в пространстве эмбеддингов = семантическая близость

В 2026 это работает на уровне предложений, не только слов



Cosine similarity — мера угла между векторами; диапазон $[-1, 1]$, ближе к 1 — более похожи. Числа illustrative.^[1]

[1] воспроизводится: all-MiniLM-L6-v2 / text-embedding-3-small

15 / 35

Word2Vec в 2013 году — историческая отправная точка для разговора об эмбеддингах: первая широко известная модель, в которой выученные векторы слов показывали семантические аналогии. С тех пор поле прошло несколько поколений моделей, и в 2026 году инженерная практика опирается на эмбеддинги ****предложений и документов****, а не на эмбеддинги отдельных слов. Это качественный сдвиг: теперь мы можем сравнивать смысл целых формулировок, не выдёргивая отдельные слова. На слайде — таблица 5×5 cosine similarity для пяти коротких предложений. Парная картина типична для современных embedding-моделей. Предложения 1 и 2 — «Как настроить SSL» и «Установка HTTPS-сертификата» — синонимы в домене веб-безопасности; их cosine similarity около 0.85. Предложения 3 и 4 — «Деплой React-компонента» и «Сборка React-приложения» — оба про работу с React; cosine около 0.78. Предложение 5 — «Рецепт борща» — против любого технического даёт значения в диапазоне 0.05–0.15. Низкие, но не нулевые: даже совершенно разные домены имеют немного общей лексической структуры. Этот пример показывает главное: современный эмбеддинг захватывает смысл, а не точное совпадение строк. «SSL» и «HTTPS» — разные строки, но они близкие в эмбеддинг-пространстве, потому что в обучающем корпусе они появлялись в одних и тех же контекстах. То же касается «React» в разных формулировках задач и любых пар синонимов в техническом языке. Технический момент про cosine similarity: формально это скалярное произведение векторов, делённое на произведение их длин. Геометрический смысл — косинус угла между векторами. Если векторы сонаправлены, cosine равен 1; ортогональны — 0; противоположны — минус 1. На практике в эмбеддинг-пространствах LLM значения почти всегда лежат в диапазоне от 0 до 1, и инженер интерпретирует их как «насколько А похоже на В по смыслу». Точные пороги — выше какого значения считать документы

похожими — зависят от задачи и подбираются эмпирически. Конкретные числа на слайде — illustrative; их можно воспроизвести бесплатно через open-source модель `sentence-transformers/all-MiniLM-L6-v2`. Источники: [1] воспроизводимо: all-MiniLM-L6-v2 / text-embedding-3-small — числа illustrative; cosine близость — статистика употребления, не семантический справочник.
<https://platform.openai.com/docs/guides/embeddings> [VFY-day-of]

Эмбединги — фундамент понимания LLM

Внутри модель работает только с векторами; слова — только на входе и на выходе.



[1] Lewis et al. (2020) — RAG

16 / 35

Соберём то, что мы прошли в этом разделе, и переформулируем главное практическое следствие. Эмбединги — это не отдельный «слой» LLM, это ****фундамент****, на котором стоит всё остальное. Модель внутри работает не со словами и не со строками — она работает с векторами в высокоразмерном пространстве. Слова появляются только на самой границе: на входе они через токенизацию превращаются в id из словаря, дальше через lookup в embedding table — в векторы. И обратно: на выходе модель выдаёт распределение по векторам, из распределения сэмплируется один — это становится id токена, и через de-tokenize выходит обратно в слова. На слайде — схема в обе стороны. Вход: `Привет, мир` → токенизация → `[Прив][ет][,][ мир]` → lookup → векторы. Сами векторы попадают в основной механизм inference, который мы разберём в Разделе 3. Выход: внутренний вектор-кандидат → distribution → выбор одного токена через сэмплинг → de-tokenize → слово. Между «человеческим» вводом и «человеческим» выводом — путь полностью векторный. И это объясняет, что мы интуитивно называем «пониманием» LLM. Когда вы спрашиваете «Как настроить SSL», а потом — «Установка HTTPS-сертификата», модель отвечает похоже, потому что эти два предложения попадают в близкие точки векторного пространства, и весь внутренний механизм работает на этих векторах одинаково. То же про синонимы — «авто» и «машина» — и про cross-lingual: «клубника» и `strawberry` дают близкие эмбединги, и модель отвечает корректно независимо от того, на каком языке к ней обратились. Что важно для практики: всё, что вы делаете с LLM — построение промптов, RAG-системы, рассуждения над документами, кросс-язык — упирается в то, как эмбединги выучены. Это инженерный фундамент, и инвестиции в правильную embedding-модель для вашего домена окупаются на следующих слоях. К конкретному применению эмбедингов в системах поиска и retrieval мы вернёмся в Лекции 3 — RAG.

Источники: [1] Lewis et al. (2020) — RAG — эмбединг ловит смысл, а не строку: основа semantic search / RAG. <https://arxiv.org/abs/2005.11401>

Раздел 3

Механизм внимания

Как модель решает, что важно сейчас

0 Введение

1 Токены

2 Эмбединги

3 **Внимание**

4 Сэмплинг

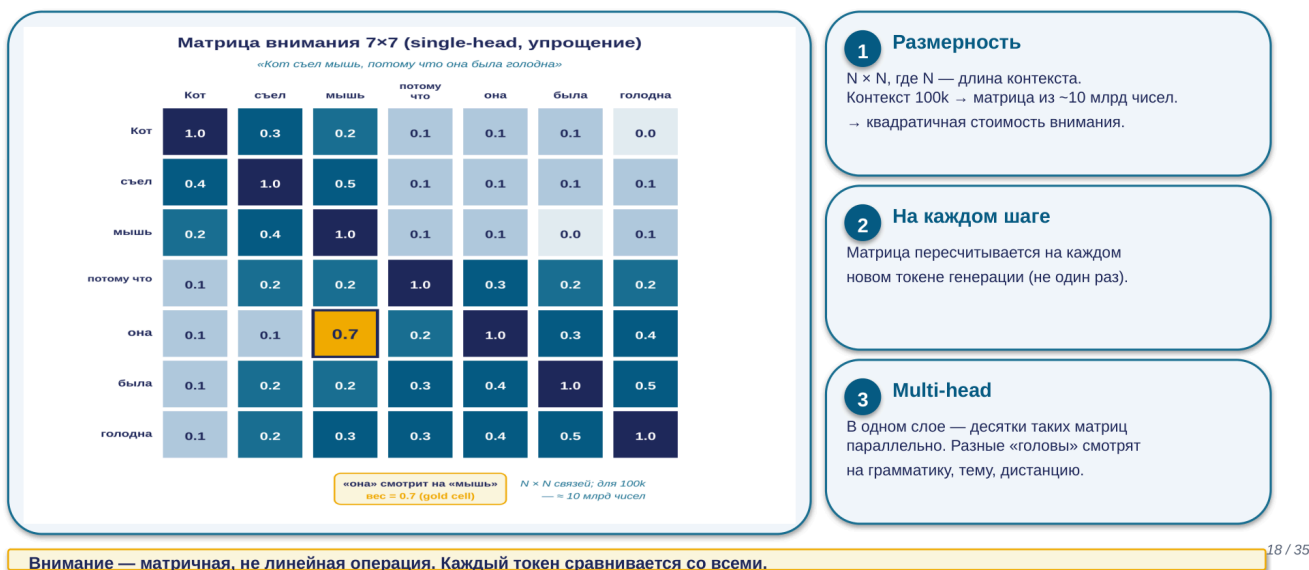
5 Финал

17 / 35

Мы прошли два этапа конвейера inference. Первый — токенизация: текст разрезается на токены, каждый получает id из словаря модели. Второй — эмбединги: каждому id ставится в соответствие выученный вектор, и геометрическая близость в этом пространстве отражает смысловую близость. Раздел 3 — про третий этап и про самый плотный для понимания механизм лекции: **механизм внимания**, или **attention**. На вход у нас уже есть последовательность эмбедингов токенов; на выходе нужно решить, на какие из них модель опирается сильнее всего, когда генерирует следующий токен. Именно это «решение, на что смотреть» делает механизм внимания. От него зависит, почему промпт с ролью работает лучше пустого, что такое контекстное окно и какие у него ограничения, почему важная информация в середине длинного промпта систематически теряется. В этом разделе четыре слайда плюс этот разделитель. Сначала мы зафиксируем базовое определение через метафору фонарика в тёмной комнате — без формул, без матриц Q/K/V, без многоголового внимания. Потом разберём рабочий пример на конкретном предложении и из него выведем эффект роли в промпте — первое из трёх «почему» Лекции 1. Затем поговорим про контекстное окно как физический предел и про квадратичную стоимость внимания. И завершим раздел разбором эффекта «lost in the middle» — почему большое окно не значит хорошее использование контекста. Раздел — самый концептуально насыщенный в лекции. Это сделано осознанно: механизм внимания — самая плотная по содержанию часть, и её стоит проходить медленно. Один section divider на всю лекцию — здесь — чтобы дать паузу перед самым плотным куском материала.

Внимание — это сверка каждого токена со всеми остальными

Каждый токен «смотрит» на все остальные одновременно. На каждом шаге — $N \times N$ связей.



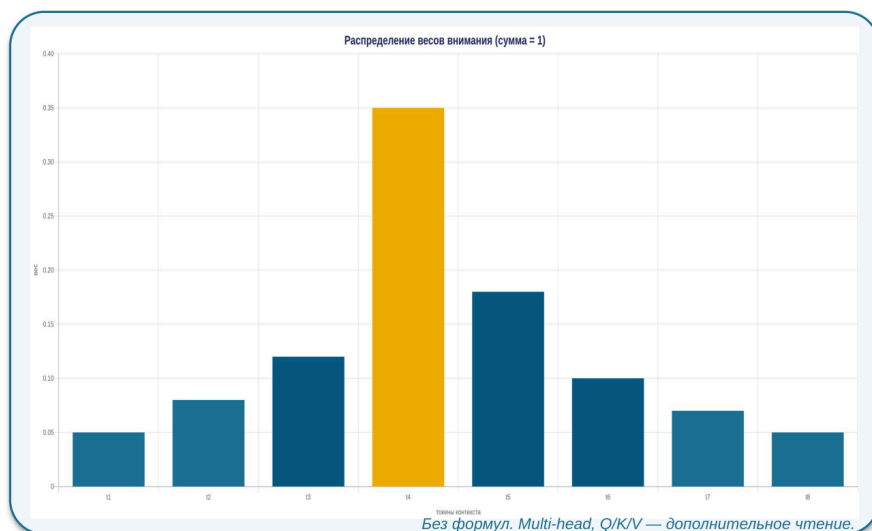
Внимание — матричная, не линейная операция. Каждый токен сравнивается со всеми.

В предыдущем слайде раздела — заголовке — мы сказали, что внимание выдаёт «распределение весов на все токены контекста». Этот слайд раскрывает важную деталь: внимание — не линейная операция, а **матричная**. Это значит, что каждый токен в контексте смотрит на каждый — и для контекста длины N матрица имеет размер $N \times N$. Эта деталь принципиальна для понимания того, что будет дальше про context window. На слайде слева — упрощённая 7×7 матрица для предложения «Кот съел мышь, потому что она была голодна». Каждая строка — это «один токен запроса смотрит на все остальные», каждый столбец — «насколько именно этот токен влияет». Числа условные, illustrative — реальные веса внимания выглядят сложнее, и сумма по строке должна равняться единице (нормировка через softmax). Но качественно картина типична: токен «она» имеет наибольший вес — 0.7 — на токен «мышь». Это и есть, что мы интуитивно называем «модель поняла, на что ссылается местоимение». Технически — это статистическая связь, выученная на огромном корпусе текстов, в которых местоимения женского рода чаще всего относятся к ближайшему существительному женского рода. Три важных следствия от того, что внимание — это матрица. Первое — **квадратичная стоимость**. Для контекста из 100 000 токенов матрица содержит 10 миллиардов чисел. Это объясняет, почему длинные контексты дорогие: даже линейное увеличение длины контекста даёт квадратичное увеличение объёма вычислений и памяти под внимание. Эту тему мы разовьём через слайд — про context window. Второе — **пересчёт на каждом шаге**. LLM генерирует токен за токеном, и на каждой новой генерации матрица внимания заново строится на текущем расширенном контексте. Это не «один раз посчитали — пользуемся весь генерацию»; это «на каждый новый токен — новая матрица». Третье — **multi-head**. На самом деле в одном слое внимания работает не одна матрица, а десятки параллельно. Это называется multi-head

attention (многоголовое внимание). Разные «головы» смотрят на разные паттерны: одна может фокусироваться на грамматических связях, другая — на тематической близости, третья — на длинных зависимостях. Разделение работы между головами — одна из ключевых архитектурных идей трансформера. Для пользователя LLM это деталь, но она объясняет, почему механизм внимания справляется с разными типами зависимостей одновременно — потому что под капотом параллельно работает несколько разных «взглядов» на одни и те же токены.

Внимание выдаёт распределение весов на все токены контекста (сумма = 1)

Какие токены сейчас важны для предсказания следующего?



[1] Vaswani et al. (2017) — Attention Is All You Need

Без формул. Multi-head, Q/K/V — дополнительное чтение.

3 свойства

- 1 На вход — все токены контекста.
- 2 На выходе — распределение, $\Sigma = 1$.
- 3 Пересчитывается на каждом шаге.



19 / 35

Третий этап конвейера — механизм внимания. На входе у нас уже есть последовательность эмбеддингов токенов; на выходе нужно решить, на какие из них модель опирается сильнее всего, когда генерирует следующий токен. Именно это «решение, на что смотреть» делает механизм внимания. Удобная метафора — **«фонарик в тёмной комнате»**. Представьте, что вы стоите в комнате с большим количеством предметов — это все токены контекста, — и вам нужно ответить на конкретный вопрос (предсказать следующий токен). Вы не можете одновременно ярко осветить всё; вы направляете фонарик на те предметы, которые сейчас релевантны вопросу. То, что в центре луча, видно ярко; то, что на периферии, — тускло. Метафора удерживается в одном слове: **внимание** — это распределение света по сцене, и это распределение меняется в зависимости от того, что мы сейчас спрашиваем. Формально на каждом шаге для каждой позиции механизм внимания возвращает распределение весов на все остальные токены контекста. Сумма весов равна единице. Один большой вес означает «я сильно опираюсь на этот токен»; маленький вес — «этот токен сейчас мне почти не важен». Никакой явной формулы здесь приводить не будем — для понимания пользователя достаточно зафиксировать три факта: на вход механизм внимания получает все токены контекста (не один и не часть, а все, сколько их есть); на выходе для каждого токена есть распределение весов с суммой 1; это распределение пересчитывается на каждом шаге генерации. В реальной модели механизм внимания устроен сложнее: в каждом слое работает не один, а несколько параллельных «голов» внимания (multi-head attention — многоголовое внимание, типично 32–128 голов в современных моделях), и каждая голова смотрит на свой аспект — кто-то ловит грамматические связи, кто-то семантические, кто-то длинные далекодействующие зависимости. Слоёв в современной модели — десятки и сотни. Для нашего уровня детализации эти

числа — справочный факт; полная техническая конструкция с матрицами Q, K, V, скейлингом через корень из размерности, маскированием и residual connections — выходит за рамки лекции; полная картина — в каноническом источнике Vaswani et al. (2017). Источники: [1] Vaswani et al. (2017) — Attention Is All You Need — attention выдаёт распределение весов на токены контекста ($\Sigma=1$); 32–128 голов на слой. <https://arxiv.org/abs/1706.03762>

Role-токены получают повышенный вес в весах внимания

Часть А — рабочий пример; часть В — эффект роли (1-е из 3 «почему»)

А. Разбор примера — куда смотрит «она»

«Кот съел мышь, потому что она была голодна»

она → мышь

толстая · главный вес

Упрощение: реальная карта внимания — сотни связей. Модель смотрит статистически, не делает грамматический разбор.

она → была

средняя

она → голодна

тонкая

Без роли

«Объясни асинхронность»

→ обобщённый ответ

(низкий вес role-токенов в весах внимания)

С ролью

«Ты **эксперт по Python**. Объясни асинхронность **джуниору**.»

→ **role-токены подсвечены**

(высокий вес в весах внимания)

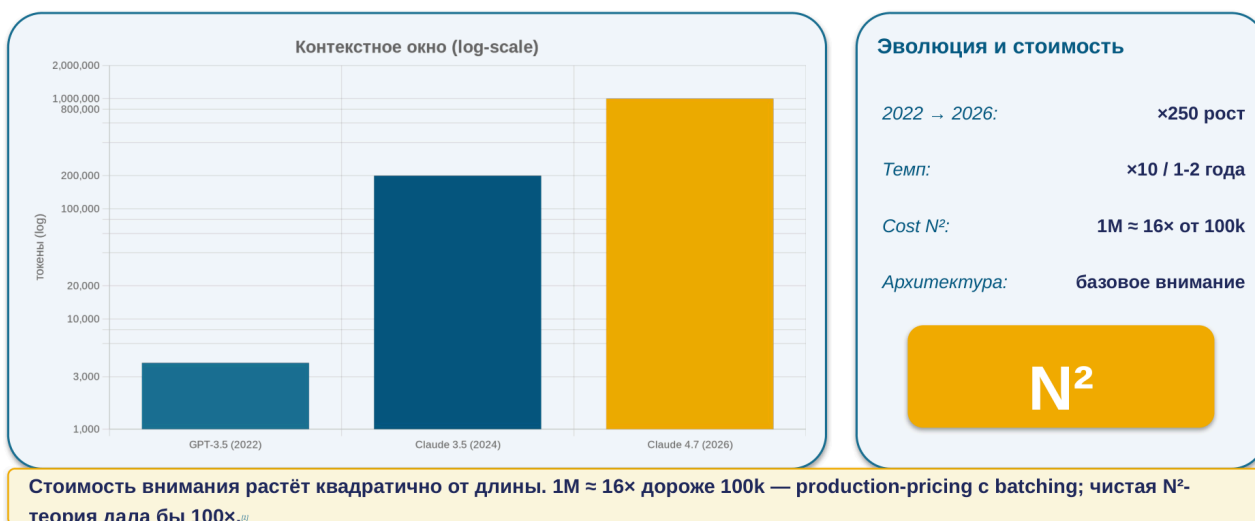
20 / 35

Возьмём конкретное предложение: «Кот съел мышь, потому что она была голодна». Когда модель доходит до токена «она», ей нужно определить, к чему этот токен относится. На уровне внимания это видно: над токеном «она» оказывается распределение весов, в котором наибольший вес — на токене «мышь» (по контексту так предложение и понимается), средний — на «была», тонкий — на «голодна». Если визуализировать, это три стрелки разной толщины. Важная оговорка: картинка с тремя стрелками — сильное упрощение. Реальная карта внимания содержит сотни связей сразу, и в каждом из десятков слоёв рисунок свой. Когда мы говорим «модель смотрит на мышь», мы агрегируем сотни значений в одну толстую стрелку. И ещё важнее: модель не делает грамматический разбор. Она статистически смотрит на токены, для которых статистика говорит «вот эти обычно связаны с "она" в подобных контекстах». Корреляция, не парсинг. Маленькое упражнение: куда смотрит модель в «Программа упала, потому что она забыла обработать null»? На большинстве современных моделей внимание здесь даёт максимум на токене «программа» — это согласуется и с грамматикой, и со статистикой технических текстов. Теперь — главное практическое следствие, отвечающее на первое из трёх «почему» Лекции 1. Сравним два промпта. «Объясни асинхронность» — без роли. «Ты эксперт по Python. Объясни асинхронность джуниору» — с ролью. На уровне внимания второй даёт качественно другую картину. В первом случае контекст короткий, и модель опирается на самую общую статистику. Во втором в контексте есть токены «Python», «эксперт», «джуниор», и они получают существенный вес в распределении внимания. Следующий генерируемый токен выбирается из распределения, смещённого этими токенами: ответ окажется более конкретным, с более простыми объяснениями, в более экспертном регистре. Рабочее объяснение: role-токены получают повышенный вес в распределении внимания

при генерации первых токенов ответа. Роль в промпте — не просьба «поверь мне», а явный input-сигнал, прямо влияющий на распределение внимания. Это первый из трёх вопросов «почему», поставленных в начале лекции, и теперь мы объясняем его через механизм.

Контекстное окно — физический предел того, сколько модель видит одновременно

Эволюция контекстного окна + квадратичная стоимость внимания



[1] Алгоритм — контекстное окно моделей Claude

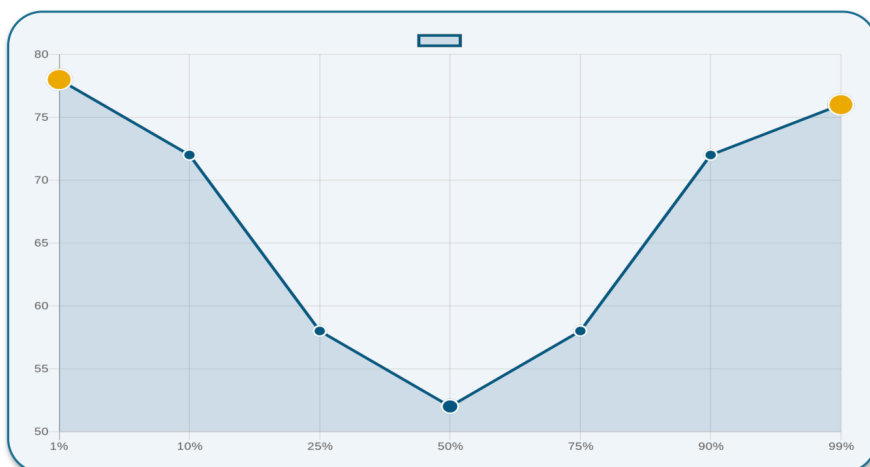
21 / 35

Цифры приведены на май 2026 года; темп роста контекстного окна — примерно ×10 каждые 1-2 года. Перед лекцией стоит проверить актуальные значения для GPT-5, Claude 4.7 и Gemini. Механизм внимания работает на всех токенах контекста, что приводит к естественному ограничению: контекстное окно — это максимальное количество токенов, которое модель может обработать за один запрос. Лекция 1 уже вводила понятие на уровне «когда история чата плюс новое сообщение перестают помещаться, старые сообщения выпадают». Сейчас мы углубляем: почему именно есть верхний предел и почему он стоит денег. Контекстное окно за последние три года выросло на порядки. Несколько ключевых точек на момент верификации в мае 2026. GPT-3.5 в момент релиза ChatGPT в 2022 году — около 4 тысяч токенов. Claude 3.5 в середине 2024 года — 200 тысяч токенов. Claude 4.7 и актуальные flagship-модели OpenAI в 2026 году — порядка одного миллиона токенов. Если к моменту чтения какие-то цифры уже сдвинулись — это нормально, индустрия движется быстро. Важен порядок (тысячи → сотни тысяч → миллион) и темп роста (увеличение примерно на порядок каждые один-два года), а не конкретные числа. Один миллион токенов — это, очень грубо, 1500–2000 страниц текста на английском. Кажется, что теперь модель «видит всё» и проблема исчезла. Это впечатление обманчиво по двум причинам. Первая — стоимость растёт квадратично. Базовый механизм внимания требует, чтобы каждый из N токенов посмотрел на каждый — это N² операций на каждом слое внимания. Удваиваем длину контекста — учетверяется стоимость inference. На практике инженерные оптимизации (FlashAttention, sparse attention, sliding-window, KV-кэш) снижают константу, но квадратичная зависимость остаётся доминирующей для текущих архитектур. В цене API это видно прямо: миллион токенов входа стоит порядка в шестнадцать раз дороже, чем сто тысяч, при той же по сложности задаче. Это структурное свойство ванильного

механизма внимания, и оно не зависит от того, какая именно цифра контекста на текущий момент актуальна. Любой следующий шаг отрасли — 1М, 10М, дальше — будет упираться в ту же квадратичную стенку, пока не появится принципиально другая архитектура. Вторая причина, по которой большое окно не панацея, — модель не одинаково хорошо использует все позиции в окне. Этому посвящён следующий слайд. Источники: [1] Anthropic — контекстное окно моделей Claude — рост окна 4k→200k→1M; порядок важнее точной цифры.
<https://www.anthropic.com/news/model-context-protocol> [VFY-day-of]

Большое контекстное окно ≠ хорошее использование контекста

Lost-in-the-middle effect — модель забывает середину^[1]



Эксперимент

Факт вставлен в позицию X (начало / середина / конец) 100k-контекста. Модель отвечает на факт.

Результаты

Начало: ~75%
Середина: ~50%
Конец: ~75%

*Liu et al. 2023.
Lost in the Middle.*

Инженерный вывод: важное помещайте в начало или в конец промпта, не в середину.

[1] Liu et al. (2023) — Lost in the Middle

22 / 35

В 2023 году группа из Stanford и UC Berkeley опубликовала работу с провокационным названием «Lost in the Middle: How Language Models Use Long Contexts». Эмпирический вопрос: если модель умеет принимать 100 тысяч токенов на вход, одинаково ли хорошо она использует разные позиции в этом окне? Эксперимент. В большой контекст — десятки тысяч токенов из произвольных документов — вставляется один значимый факт: в начале, в середине или в конце. Затем модель спрашивают факт и считают точность ответа. Результаты дают характерную U-образную кривую: точность около 70–80% когда факт лежит в начале, проседает до ~50% в середине, и снова поднимается до 70–80% к концу. Феномен авторы назвали «потеря в середине». Природа эффекта — в том, как модели учатся работать с длинным контекстом. На обучающем корпусе типичные документы устроены так, что важные утверждения находятся либо в начале (тезис), либо в конце (вывод); статистика положения важных токенов имеет U-форму. Модель усваивает эту статистику и переносит её на inference: даже если важный токен лежит в середине, модель «по привычке» меньше его взвешивает во внимании. Это не баг — это inductive bias обучения, отражение того, как устроены реальные тексты. Инженерный вывод. Если у вас длинный промпт с инструкциями или данными — самое важное помещайте в начало или в конец, не в середину. Частая ошибка — длинная преамбула с критическим ограничением, утопленным посередине; такую инструкцию модель будет систематически игнорировать. Решение: переписать промпт так, чтобы критические инструкции стояли в самом начале (system prompt) или были повторены явно в конце прямо перед задачей. Это объясняет также, почему «просто засунуть всю документацию в большое окно» — плохая стратегия для RAG. Хороший retrieval, возвращающий 5–10 целевых фрагментов в начале промпта, систематически работает лучше, чем загрузка всей базы в гигантское

окно. К этому вернёмся в Лекции 3. Источники: [1] Liu et al. (2023) — Lost in the Middle — U-shape: точность проседает в середине окна; важное — в начало/конец промпта. <https://arxiv.org/abs/2307.03172>

Раздел 4

Сэмплинг

От распределения к токену

0 Введение

1 Токены

2 Эмбединги

3 Внимание

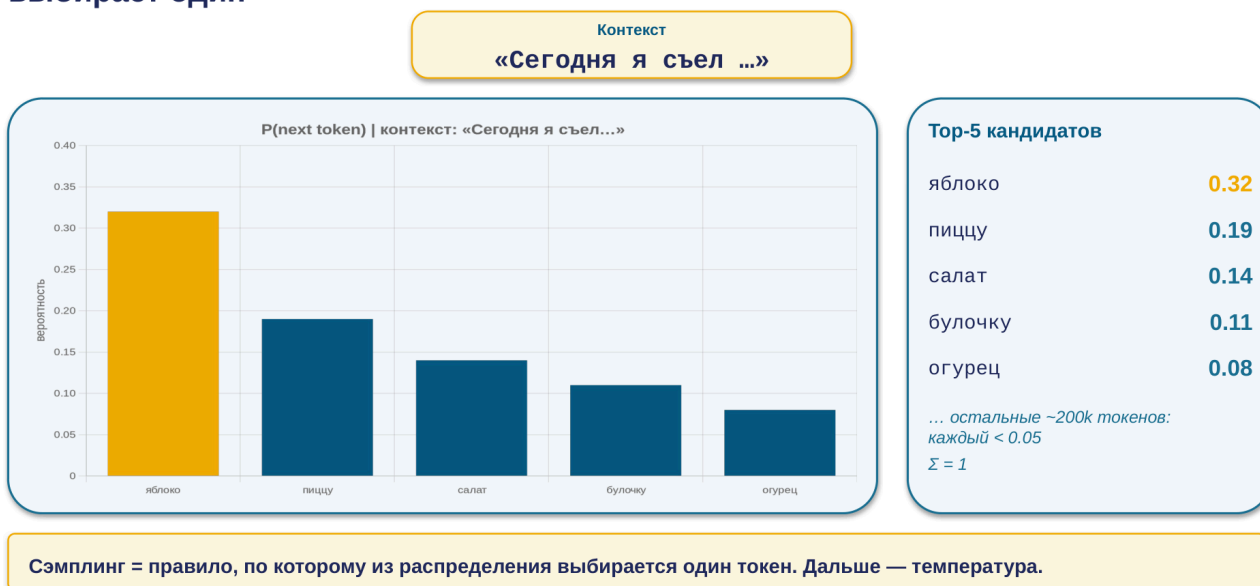
4 Сэмплинг

5 Финал

23 / 35

Мы закрыли раздел про внимание: модель умеет распределять важность по токенам контекста. Но на этом inference ещё не закончен. У модели есть промежуточный внутренний результат — распределение вероятностей на все ~200 тысяч токенов словаря, — и она должна из этого распределения выбрать ровно один токен, который и станет следующим в ответе. Этот шаг и называется сэмплингом. От него зависит то, что воспринимается пользователем как «стиль» модели: насколько детерминированно она отвечает на одинаковые запросы, насколько креативна или скучна, насколько предсказуема. И именно сэмплинг закрывает третье из трёх «почему» Лекции 1: почему один и тот же запрос даёт разные ответы. В этом разделе пять слайдов. Сначала зафиксируем, что такое распределение вероятностей на конкретном примере с предсказанием следующего токена. Потом разберём главную ручку API — температуру — на трёх режимах: $T=0$ (детерминированный argmax), $T \approx 1$ (стандарт для чата), $T=2$ (хаос). Дальше — практическая матрица из четырёх параметров API под четыре сценария: классификация, кодогенерация, чат-объяснение, творческое письмо. Затем покажем сэмплинг в контексте полного цикла авторегрессионной генерации — где он стоит между двумя forward-проходами модели. И завершим разделом сравнением inference локально и в облаке: тот же конвейер, но разный размер модели и среда.

На каждом шаге модель выдаёт распределение вероятностей на все токены — выбирает один



24 / 35

Перейдём к четвёртому, последнему этапу inference: сэмплингу. На входе сэмплинга — то, что произвели предыдущие три этапа: модель прошла токенизацию, эмбединги, все слои внимания, и на выходе у неё распределение вероятностей на все токены словаря. То есть для каждого из ста или двухсот тысяч токенов словаря модель сказала: «вероятность того, что следующим окажется именно этот, равна такому-то значению». Сумма всех вероятностей равна единице. Возьмём конкретный пример. Пользователь написал в чат «Сегодня я съел ...» и ждёт продолжения. Внутри модели появляется распределение вероятностей следующего токена. На бар-диаграмме это выглядит примерно так: `яблоко` — 0.32, `пиццу` — 0.19, `салат` — 0.14, `булочку` — 0.11, `огурец` — 0.08, и остальные двести тысяч токенов — каждый с вероятностью меньше 0.05. Числа на слайде иллюстративные; конкретные значения зависят от модели и от контекста, но общая картина устойчива. Видно две вещи. Первая — распределение ****не равномерное****: модель имеет статистические предпочтения, основанные на том, какие токены часто следовали за «съел» в обучающем корпусе. Это и есть «знание» модели — статистика следующего токена для каждого возможного контекста. Вторая — распределение ****не точечное****: есть несколько правдоподобных кандидатов, и среди них вероятности заметно различаются. Модель не «знает» один правильный ответ; у неё есть множество кандидатов с убывающими вероятностями. Дальше начинается сэмплинг. Сэмплинг — это правило, по которому модель из этого распределения выбирает один токен, который и пойдёт в ответ. От правила сэмплинга зависит, насколько «творческим» или «детерминированным» будет ответ. На следующем слайде разберём главный параметр сэмплинга — температуру — и попутно затронем top-p и top-k как альтернативные ручки тонкой настройки. Заметьте: даже на этом слайде уже видно ключевое условие

того, что один и тот же запрос может давать разные ответы — есть распределение, из которого что-то выбирается. Это и есть третье из трёх «почему» Лекции 1, и сейчас мы аккуратно раскроем его механизм.

Температура: насколько острым будет выбор

$T = 0$ (argmax) · $T = 1.0$ (стандарт) · $T = 2.0$ (хаос)



Альтернативные ручки: top-p (nucleus) — отрезает редкие токены по Σ ; top-k — по числу кандидатов. Достаточно T для start. ¹¹

[1] Holtzman et al. (2019) — nucleus sampling (top-p)

25 / 35

Главный параметр сэмплинга — температура, обозначается T. Температура управляет, насколько «острым» будет выбор. Это параметр API: ``temperature=0.0``, ``temperature=0.7``, ``temperature=2.0``. Технически температура — это коэффициент, на который делят логиты (внутренние «сырые» оценки модели до нормализации в вероятности) перед операцией нормализации; вводить формулу здесь не будем, важно следствие. При $T = 0$ модель выбирает токен с самой высокой вероятностью. Это операция `argmax` — «возьми тот, у которого вероятность максимальна». В нашем примере про «съел ...» модель практически всегда выберет `яблоко` (0.32 — максимум). Ответ предсказуем и почти детерминированный — повторите запрос десять раз, получите одно и то же. Маленькая оговорка: в production может быть микро-вариативность из-за того, что серверный батчинг и floating-point на GPU не дают побитово одинаковых результатов между запусками; для большинства задач эта вариативность игнорируема. При $T = 1.0$ — стандартный режим. Модель сэмплирует пропорционально исходным вероятностям: `яблоко` будет выбрано примерно в 32% случаев, `пиццу` в 19%, `салат` в 14% и так далее. Это даёт естественную вариативность — каждый запуск может вернуть разный ответ, но все ответы лежат в зоне правдоподобных. При $T = 2.0$ распределение «сглаживается»: разница между самыми вероятными и редкими токенами уменьшается, и модель начинает чаще выбирать неожиданные варианты. В крайних случаях ответы получаются почти хаотичными — выбираются токены, которые в реальном языке маловероятны после «съел». Кроме температуры, есть две альтернативные ручки: top-p (nucleus sampling) — параметр, отрезающий «хвост» редких токенов: модель оставляет минимальное подмножество самых вероятных токенов, у которого суммарная вероятность не меньше p, и сэмплирует только из них. И top-k — параметр-предел количества кандидатов: модель оставляет top-k самых

вероятных токенов. На практике для большинства задач достаточно настраивать температуру; top-p и top-k — это второй слой контроля. Этот слайд отвечает на третий из трёх вопросов «почему», поставленных в начале лекции: почему один и тот же запрос даёт разные ответы. Ответ короткий: потому что при T больше нуля сэмплинг — стохастический процесс, и из одного и того же распределения каждый запуск может выбрать разный токен. Это не баг; это инженерное решение, дающее моделям естественную вариативность, что для большинства интерактивных задач воспринимается как «человекоподобность». Источники: [1] Holtzman et al. (2019) — nucleus sampling (top-p) — температура и top-p управляют формой распределения при сэмплинге. <https://arxiv.org/abs/1904.09751>

4 ручки API под задачу: temperature, top_p, max_tokens, system prompt

Подобрать параметры под сценарий обоснованно

Сценарий	temperature	top_p	max_tokens	system_prompt
Классификация / точное извлечение	0	—	50–200	Минимальный, со схемой выхода
Кодогенерация	0.2–0.3	0.9	1000+	Роль + контекст репозитория
Чат-объяснение пользователю	0.7	0.9	500–1000	Роль + описание аудитории
Творческое письмо	0.9–1.2	0.95	2000+	Роль + описание стиля

T = 0 практически детерминирует выбор; в production возможна микро-вариативность из-за batching — для большинства задач игнорируема.

26 / 35

Соберём четыре основных параметра, которыми инженер управляет работой LLM через API. Эти четыре параметра — `temperature`, `top\_p`, `max\_tokens`, **системный промпт** — ключевая практическая цель этого раздела: после этого слайда вы должны уметь подобрать их под конкретный сценарий, не наугад, а с обоснованием. Краткое определение каждого. **Temperature** — острота сэмплинга, от 0 до примерно 2. **Top_p** — параметр nucleus-отрезания (типичные значения 0.9–0.95). **Max_tokens** — максимальная длина ответа модели в токенах (не в словах); если ответ упирается в лимит, он обрезается на полуслове. **Системный промпт** — отдельное сообщение, задающее роль и ограничения модели до пользовательского запроса. Сводная таблица под четыре типовых сценария. Логика последовательная. **Классификация.** Когда задача — присвоить документу одну из конечного набора меток, любая стохастичность вредна: если на одном и том же документе модель в одном запуске говорит «жалоба», а в другом — «вопрос», воспроизводимость нарушена. $T = 0$ обеспечивает детерминированный argmax . Max_tokens маленький — ответ короткий. Системный промпт минимальный, но со схемой выхода: «отвечай в формате JSON `{label: ..., confidence: ...}`», чтобы постпроцессинг легко парсил результат. **Кодогенерация.** Здесь нужна слегка ненулевая температура. $T = 0$ даёт повторяющийся, иногда чрезмерно «учебниковый» код; $T = 0.2\text{--}0.3$ оставляет вариативность только в выборе между почти одинаково хорошими решениями. $\text{Top_p} = 0.9$ отсекает редкие токены, которые в коде почти всегда означают синтаксическую ошибку. Max_tokens большой. Системный промпт — роль и контекст репозитория. **Чат-объяснение.** Стандартный режим. $T = 0.7$ даёт разнообразие формулировок при сохранении смысла; $\text{top_p} = 0.9$ оставляет естественные варианты. Системный промпт описывает роль и аудиторию. **Творческое письмо.** Стохастичность — желаемое свойство. $T = 0.9\text{--}1.2$ даёт

неожиданные обороты и метафоры. $\text{top_p} = 0.95$ ослабляет отсечение «хвоста». Системный промпт — роль и стиль. Эта таблица — не предписание; она ориентир. В реальной работе вы будете калибровать значения под свою задачу. Важно понимать рамку: четыре ручки, и они дёргаются согласованно — нет смысла ставить $T = 0$ и $\text{top_p} = 0.5$ (top_p при $T = 0$ не имеет смысла); нет смысла ставить $T = 1.5$ и $\text{max_tokens} = 50$ (модель только разогнётся, и её обрежет).

Цикл: предсказали токен → добавили в контекст → предсказываем следующий

Авторегрессионная генерация — длинный ответ из stateless-вызовов



Каждый шаг — stateless. «Память» одного ответа несёт сам контекст, не модель.

27 / 35

Соберём четыре этапа inference в единый цикл. Лекция 1 описывала модель как stateless-функцию: «вход — данные, выход — предсказание, между вызовами состояния нет». Но в чате с LLM мы наблюдаем не одну функцию, а длинный ответ — фразу, абзац, страницу. Откуда берётся длинный ответ из stateless-вызовов? Ответ — авторегрессионная генерация. От латинского «само-регрессирующий»: модель опирается на свои предыдущие выводы. Это циклический процесс из пяти шагов. Первый — текущий контекст: системный промпт плюс история диалога плюс новый запрос плюс всё, что модель уже сгенерировала за этот ответ. Второй — forward pass: полный проход всего контекста через четыре этапа inference, которые мы разобрали в первых трёх разделах лекции — токенизация, эмбеддинги, внимание через все слои, финальный слой. На выходе forward pass — распределение вероятностей следующего токена. Третий — сэмплинг: по правилу температуры и top-p из распределения выбирается один токен. Четвёртый — добавление в контекст: выбранный токен дописывается к ответу и становится частью контекста для следующего шага. Пятый — возврат к шагу 1. Цикл повторяется, пока модель не сгенерирует специальный токен «конец ответа» или пока не упрётся в `max\_tokens`. Каждый отдельный шаг этого цикла — stateless: модель ничего не «помнит» между шагами; всю «память» несёт сам контекст, который каждый раз подаётся целиком. Это уточнение к тому, что говорила Лекция 1: модель действительно stateless, но из stateless-шагов собирается процесс, имеющий вид связного ответа, за счёт того, что каждый следующий шаг видит весь предыдущий контекст. Иллюзия «памяти» внутри одного ответа создаётся не моделью, а оркестратором, который собирает и подаёт контекст. Это объясняет несколько практических наблюдений. Скорость генерации: длинный ответ модель пишет по одному токenu за раз, десятки токенов в секунду; длинные

ответы видны как «текст печатается на ваших глазах» — это физический темп генерации. Поведение ``max_tokens``: когда счётчик токенов достигает лимита, цикл обрывается мгновенно — на полуслове, на середине JSON. Стриминг: API современных LLM поддерживают режим, в котором токены возвращаются клиенту по мере генерации, не пакетом в конце. Терминологически: каноническая русская форма — «авторегрессионный» (по модели «регрессионный»). Форма «авторегрессивный» допустима, но менее распространена в современной русской документации.

Inference одинаков локально и в облаке — но размер модели определяет качество

Архитектурно — тот же конвейер. Различия — в размере и среде.

Local (Ollama, llama.cpp, LM Studio)

Размер: 1–13B параметров

- Qwen 2.5 1.5B · Llama 3.2 1B
- Llama 3.1 8B · Mistral 7B^[1]

- **Приватность** *запросы не уходят провайдеру*
- **Скорость** *медленнее на consumer hardware*
- **Контекст** *ограниченное окно*
- **Цена** *0 за токен (своё железо)*

Cloud (OpenAI, Anthropic, Yandex, Сбер)

Размер: 200B+ параметров

- GPT-5, Claude 4.7
- YandexGPT, GigaChat
- Gemini

- **Качество** *лучше на сложных задачах*
- **Задержка** *200–500 мс*
- **Контекст** *до 1M токенов*
- **Цена** *оплата за токены, RU ≈ 2× EN*

[1] Mistral AI — Mistral 7B

Описанный выше цикл inference — токенизация, эмбединги, внимание, сэмплинг, возврат к началу — один и тот же в облачных LLM-сервисах и в локально развёрнутых моделях. С точки зрения механизма ChatGPT, Claude, GigaChat, локальная Llama 3.1 на ноутбуке или Qwen 2.5 на edge-устройстве работают одинаково. Различаются они размером модели (и, как следствие, качеством ответов) и средой исполнения. Локальное развёртывание — Ollama, llama.cpp, LM Studio, vLLM — обычно работает с моделями размером 1–13 миллиардов параметров: Qwen 2.5 1.5B, Llama 3.2 1B, Llama 3.1 8B, Mistral 7B на 2026 год. Преимущества — приватность данных (запросы не уходят к стороннему провайдеру), отсутствие платы за токен, независимость от интернета. Недостатки — скорость на consumer-hardware заметно ниже, чем у облачных провайдеров; контекстное окно у локальных моделей часто меньше; качество ответов на сложных задачах уступает большим моделям. Облачные сервисы — OpenAI, Anthropic, Google, российские Yandex и Сбер — работают с моделями значительно большего размера (порядка сотен миллиардов параметров и больше). Качество ответов на сложных задачах — выше; контекстное окно — больше (сотни тысяч и миллион токенов); типичная задержка ответа — 200–500 мс. Платится это передачей данных через API провайдера (со всеми вопросами политики хранения и использования, которые мы разбирали в Лекции 1) и оплатой за токены — которая для русского текста выше, чем для английского, как мы видели ранее сегодня. Глубокая дискуссия о локальных и облачных моделях — со схемой принятия решения, точкой безубыточности по объёму запросов, регуляторными аспектами — была в Лекции 1 и здесь не повторяется. Сюда важно унести один тезис: архитектурно inference выглядит одинаково; различия — в размере модели и в среде. Когда вы выбираете между «локально» и «в облаке», вы выбираете не разные технологии, а разные точки на шкале

«качество × приватность × стоимость», и эта шкала не зависит от того, как именно устроен цикл inference внутри. Источники: [1] Mistral AI — Mistral 7B — локальные open-weight модели 1-13B vs cloud 200B+.
<https://mistral.ai/news/announcing-mistral-7b> [VFY-day-of]

Раздел 5

Финал

Заккрытие 3 «почему» + мост к Лекции 3

0 Введение

1 Токены

2 Эмбединги

3 Внимание

4 Сэмплинг

5 Финал

29 / 35

Мы прошли все четыре этапа inference: токенизация, эмбединги, внимание, сэмплинг. Чёрный ящик из Лекции 1 параграфа 3.2, который мы тогда называли «inference моделью», перестал быть чёрным. Дальше — финальный раздел лекции, в котором мы собираем картину воедино и закрываем цикл. В этом разделе шесть слайдов. Сначала — короткий гесар: те же четыре этапа теперь сложены в горизонтальный конвейер; одна строка определения под каждой стадией. Потом — главный практический итог лекции: явные ответы на три «почему» из Лекции 1. Почему промпт с ролью работает лучше пустого. Почему AI плохо считает буквы. Почему один и тот же запрос даёт разные ответы. На каждом «почему» ставим явную ссылку на механизм, который мы только что разобрали. Дальше — три кросс-режущих фрейма. Первый — дерево решений «когда не LLM»: ситуации, в которых классический ML, прозрачные методы или специализированная маленькая модель работают лучше большого языкового. Второй — концептуальная разница между вниманием как корреляционным механизмом и человеческой моделью причинности; callback на трёхуровневую иерархию Перла из Лекции 1. После этого — задание к Семинару 2 и краткий тизер Лекции 3 про агенты, RAG и MCP. И в финале — Q&A на вопросы в зале.

4 этапа inference сложились в конвейер

Тот же чёрный ящик из Лекции 1 — теперь распакован



Лекция 1 называла этот конвейер «inference моделью» — чёрным ящиком. Теперь он перестал быть чёрным.

30 / 35

Сложим всё, что мы разобрали, в одну схему. Конвейер inference LLM состоит из четырёх этапов. **Первый — токенизация.** Текст пользователя разрезается на токены — id из фиксированного словаря модели. Словарь построен один раз до обучения BPE-алгоритмом. Конкретное разбиение зависит от частотности подпоследовательностей в обучающем корпусе. **Второй — эмбеддинг.** Каждому токenu сопоставлен выученный вектор из embedding-таблицы. Геометрическая близость векторов отражает семантическую близость токенов в обучающем корпусе. **Третий — механизм внимания.** Через десятки слоёв внимания модель строит распределения весов на токены контекста — какие токены сейчас важны для предсказания следующего. На выходе — распределение вероятностей на все токены словаря. **Четвёртый — сэмплинг.** Из этого распределения по правилу, заданному температурой и top-p / top-k, выбирается один токен. Он добавляется в контекст, и цикл повторяется до конца ответа. Этот конвейер — то, что Лекция 1 называла «inference моделью». Тогда мы оставили его как чёрный ящик; теперь он перестал быть чёрным. Полезно зафиксировать: каждый из четырёх этапов имеет хотя бы одно прямое следствие на уровне инженерной практики. Токенизация даёт слепоту к буквам, разную стоимость на разных языках, требование внешнего инструмента для точной посимвольной работы. Эмбеддинг даёт семантический поиск, базовый слой RAG, кластеризацию. Механизм внимания даёт эффект роли в промпте, контекстное окно как ограничение, «lost in the middle». Сэмплинг даёт стохастичность ответа, четыре ручки API под сценарий, авторегрессионную природу длинного ответа. Эти восемь следствий — основное полезное содержание лекции; если на них вы можете опереться при принятии решений в своей инженерной работе, главное усвоено.

Ответы на вопросы из начала лекции

1

Почему промпт с ролью работает лучше пустого?

На уровне внимания *role*-токены получают высокий вес — модель опирается на них при выборе следующих токенов.



2

Почему AI плохо считает буквы?

Токенизатор объединяет буквы в токены. *strawberry* — 3 токена, не 10 букв. Модель видит токены, не буквы.

01
10

3

Почему один и тот же запрос даёт разные ответы?

Сэмплинг — стохастический выбор из распределения при $T > 0$. Каждый запуск может выбрать разный токен.



31 / 35

В начале сегодняшней лекции мы поставили три вопроса «почему» — и обещали ответить на них не интуицией, а конкретным механизмом. Теперь у нас есть всё нужное, чтобы это сделать. ****Первое «почему»:** почему промпт с ролью работает лучше пустого?** На уровне механизма внимания *role*-токены — «ты эксперт по X», «отвечай как Y» — получают существенный вес в распределении внимания. Модель опирается на них при выборе следующих токенов; выбор оказывается смещённым в сторону, согласованную с ролью. Это упрощённое объяснение; альтернативно тот же эффект объясняется через *in-context steering* и через эффекты RLHF-обучения. В любом случае: роль в промпте — это не просьба «поверь мне», а конкретный *input*-сигнал, влияющий на распределение внимания. ****Второе «почему»:** почему AI плохо считает буквы?** Потому что модель видит не буквы, а токены. Слово ``strawberry`` для неё — три токена ``[st][raw][berry]``, а не десять букв; внутри каждого токена нет явного перечисления букв на позициях. Слепота к буквам — структурное следствие BPE-токенизации, и оно не исправляется ни *fine-tuning*, ни более крупной моделью на чистом *inference*. Для точной посимвольной работы нужен внешний инструмент: *Code Interpreter*, *Python*, регулярное выражение. ****Третье «почему»:** почему один и тот же запрос даёт разные ответы?** Потому что при *temperature* больше нуля сэмплинг — стохастический процесс, выбирающий один из правдоподобных кандидатов из распределения вероятностей следующего токена. Каждый запуск может выбрать иначе. При *temperature* равной нулю ответ практически детерминированный, с микро-вариативностью из-за серверного батчинга; при единице — естественно вариативный; при больше единицы — всё более хаотичный. Эти три ответа — главный практический итог сегодняшней лекции. Если вы можете назвать механизм для каждого «почему» в одном-двух предложениях, главное содержание усвоено. Заметьте: ответы не интуитивные («модель такая, потому

что её плохо обучили»), а механистические — каждое «почему» прицельно сводится к конкретному внутреннему этапу конвейера. Эта механистичность — то, что отличает инженерное понимание LLM от пользовательского.

Когда LLM — не правильный инструмент: дерево решений



32 / 35

Расширим классификацию задача × модальность из Лекции 1 одним практическим вопросом: когда LLM — не правильный инструмент. Универсальное использование больших моделей привлекательно, но в значимом числе сценариев оно проигрывает по эффективности классическим методам ML. Простое решение в форме decision tree из трёх веток. ****Первая ветка.**** Задача — классификация на маленьком фиксированном наборе категорий: 5–20 классов, тысячи или десятки тысяч примеров с разметкой? Скорее всего, выбор — классический ML: логистическая регрессия, gradient boosting (популярные библиотеки XGBoost, LightGBM, CatBoost), маленький transformer-классификатор вроде BERT-base, донастроенный под конкретную задачу (это называется fine-tuning). LLM здесь будет более дорогой и менее точный. ****Вторая ветка.**** Нужна интерпретируемость или регулируемость — финансы, страхование, медицина, юридическая сфера, всё, где нужно объяснить регулятору, почему модель приняла такое решение? Здесь выбор — классические методы с прозрачной структурой: логистическая регрессия с feature importance, decision trees, rule-based системы. LLM в смысле объяснимости отдельного предсказания — «чёрный ящик». ****Третья ветка.**** Критическое время отклика — меньше 100 миллисекунд? Real-time персонализация ленты, антифрод на платёжном шлюзе, edge-устройства? Здесь — специализированная маленькая модель (классический ML или маленькая нейросеть), а не вызов LLM с задержкой 200–500 миллисекунд. Во всех остальных случаях LLM применима, и часто оптимальна. Особенно для задач, требующих обработки естественного языка, гибкого формата запроса, многошагового рассуждения, генерации новых текстов или кода. Эта схема — верхний уровень. Реальный инженерный выбор глубже, и в курсе мы будем возвращаться к нему многократно: Лекции 4–7 пройдут по конкретным индустриям — ПО, финансы, ритейл, медицина — и для каждой обсудим, где LLM

оптимум, а где over-engineering. Здесь важно зафиксировать одно: LLM — не универсальный молоток. Знать механику внутренностей LLM нужно в том числе для того, чтобы аккуратно понимать, где она не нужна.

Внимание статистически смотрит на токены — не понимает причинности

ИИ считает корреляции в данных, не строит каузальный граф



Человек

«X произошло, потому что Y»

Модель причинности — строит механизмы.

Опирается на физическую интуицию, доменные знания, знание механизмов мира. Для причинных выводов нужен именно этот опыт — не статистика.



ИИ (через внимание)

«X следует за Y в данных»

Статистическая корреляция, не причинность.^[1]

Замечает паттерн «X и Y часто соседствуют» в обучающих данных. Для причинных выводов — привлекайте эксперта предметной области или причинные методы.

[1] Pearl (2018) — The Book of Why

Третий короткий кросс-сюжетный фрейм — возврат к одному из самых концептуально важных мест Лекции 1: трём уровням причинности Перла. Лекция 1 показала, что современные AI-системы устойчиво работают на уровне 1 (ассоциация), частично — на уровне 2 (вмешательство), и не справляются с уровнем 3 (контрфактуальность). Теперь у нас есть механистическое основание для этого тезиса. Механизм внимания статистически смотрит на токены — он не строит каузальный граф. Когда модель видит в контексте предложение «X произошло, потому что Y», она замечает, что токены X и Y часто соседствуют в обучающем корпусе в подобных конструкциях. Она усваивает корреляцию, не причинность. Конструкция «потому что» для модели — это паттерн в данных, не указание на каузальный механизм реального мира. Сравним два процесса. Человек: «X произошло, потому что Y» — модель причинности, включающая уровни 1 (наблюдение корреляции), 2 (что произойдёт, если изменить Y) и 3 (что было бы, если бы Y не было). Эта модель опирается на физическую интуицию, на доменные знания, на знание механизмов мира. AI на основе механизма внимания: «X следует за Y в данных» — статистическая корреляция между токенами. Сильный сигнал на уровне 1; на уровень 2 модель частично переходит, если в обучающем корпусе было много примеров изменения Y; на уровень 3 в общем виде — нет. Это объясняет феномен, описанный ещё в Лекции 1: модель прекрасно отвечает на «что коррелирует с чем» (анализ данных). Хуже — на «что произойдёт, если я изменю X» (оценка вмешательства). И почти никак — на «было бы Y, если бы X не случилось» (контрфактуальный анализ). Контрфактуальные вопросы остаются зоной человеческого экспертного суждения — это не временный технический недостаток, а ограничение текущей парадигмы. Тон замечания — фактический, не алармистский. Механизм внимания — мощный инструмент в области, в которой он работает. Эта область — корреляционная

статистика над данными, и инженер обязан помнить границу при проектировании систем, в которых от AI ожидают каузальных выводов.

Практический вывод: там, где решения требуют именно причинного вывода, привлекайте эксперта предметной области или отдельные причинные методы — механизм внимания сам по себе для этого не предназначен. Источники: [1] Pearl (2018) — The Book of Why — attention ловит ассоциацию, не причинность (3 уровня причинной лестницы). <http://bayes.cs.ucla.edu/WHY/>

Лекция 3: «Агенты, RAG, API — как AI выходит за пределы чата»

Все 4 концепции надстраиваются над одним проходом inference



[1] Lewis et al. (2020) — RAG · [2] Anthropic — Model Context Protocol (25 ноя 2024) · [3] Yao et al. (2022) — ReAct

34 / 35

Тема следующей лекции — «Агенты, RAG, API: как AI выходит за пределы чата». В нашем конвейере inference, который мы только что собрали, есть жёсткое ограничение: модель видит только то, что есть в контексте, и не может пойти за информацией наружу. Лекция 3 покажет, как это ограничение обходится через четыре класса инструментов. ****RAG — Retrieval-Augmented Generation.**** Это расширение, в котором перед обращением к LLM выполняется семантический поиск по вашей базе знаний на эмбедингах — точно тех, что мы разбирали в разделе про эмбединги сегодняшней лекции — и top-K релевантных фрагментов добавляются в контекст. На уровне inference это всё тот же конвейер из четырёх этапов, только с обогащённым контекстом. RAG — самый прямой способ дать модели доступ к информации, которой нет в её весах. ****Tools или function calling.**** Механизм, через который модель генерирует специально структурированный вызов — обычно в формате JSON, иногда в других форматах в зависимости от API-провайдера — указывающий, какой инструмент она хочет вызвать и с какими параметрами. Внешняя система видит этот вызов, выполняет его, и результат возвращается модели как часть контекста. Это канонический способ обойти ограничения слепоты к буквам (для арифметики — Python sandbox) и stateless-природы (для актуальной информации — веб-поиск). ****MCP — Model Context Protocol.**** Открытый стандарт подключения инструментов к LLM, представленный Anthropic в ноябре 2024 года. Решает проблему «каждое сочетание модели и инструмента — отдельная интеграция»: один протокол, переиспользуемое подключение. Мы упоминали MCP в Лекции 1; в Лекции 3 разберём подробнее. ****Agent loop.**** Цикл act → observe → reflect, рассмотренный ещё в Лекции 1. На каждом шаге модель решает действие, видит результат, корректирует план — и так до достижения цели. Все четыре концепта надстраиваются над single-shot inference, который мы собрали сегодня.

Multimodal-расширения — CLIP-эмбединги для изображений, аудио-эмбединги, унифицированные модели типа GPT-4o, обрабатывающие текст и изображение совместно — там же, в составе RAG для нетекстовых модальностей. На семинаре будем разбирать ваши домашние эксперименты с температурой; постарайтесь успеть прогнать запросы заранее. А пока — оставшееся время мы посвящаем вашим вопросам. Источники: [1] Lewis et al. (2020) — RAG — как AI выходит за пределы чата: retrieval-augmented generation. <https://arxiv.org/abs/2005.11401> [2] Anthropic — Model Context Protocol (25 ноя 2024) — открытый стандарт подключения инструментов к LLM. <https://www.anthropic.com/news/model-context-protocol> [3] Yao et al. (2022) — ReAct — agent loop: act → observe → reflect. <https://arxiv.org/abs/2210.03629> [VFY-day-of]

Q&A

Спасибо

контакты лектора — заполняется перед лекцией

35 / 35

На этом основная часть лекции закончена. Дальше — оставшееся время отдаём вашим вопросам. Нет «глупых» вопросов и нет «слишком базовых» — если что-то прозвучало неясно или у вас в голове не сложилась картинка одного из четырёх этапов, лучше спросить сейчас, пока тема свежая. Если вопросов в зале немного, я могу зайти с другой стороны: рассказать, какие из четырёх этапов чаще всего вызывают трудности на следующих занятиях, или показать дополнительный пример токенизации на ваших словах. Если совсем тишина — это тоже нормально, и тогда напомним, что на семинар через неделю мы принесём результаты домашнего задания по температуре, прогоним их через простой чек-лист и обсудим, какую температуру стоит ставить в production. Если по ходу обсуждения возникнут вопросы — приносите их к семинару или пишите по e-mail. Спасибо за внимание и за вашу включённость в течение всей лекции. До следующей встречи.